

Síntesis, traducción y verificación de sistemas de ecuaciones utilizados en Protocolos de Conocimiento Cero

Lucía Martínez Martínez

Grado en Ingeniería Informática
Universidad Complutense de Madrid



Trabajo de Fin de Grado
Curso 2025 - 2026

Directores:

Clara Rodríguez Núñez
Alejandro Hernández Cerezo

Índice general

1. Introducción a Circom	3
1.1. Zero Knowledge Proof	3
1.2. CIRCOM	4
1.3. R1CS	6
1.4. Snarkjs	7
1.5. Descripción del problema	8
2. Desarrollo del problema	9
2.1. Ejemplo del problema	9
2.2. Notación	9
2.3. Reglas de transformación	9
2.3.1. Operadores Lógicos	10
2.3.2. Operadores de Comparación	10
2.3.3. Operadores Aritméticos	16
2.3.4. Operadores de desplazamiento	16
2.3.5. Acceso a Arrays	16
2.3.6. Expresiones condicionales	16
2.3.7. Expresiones condicionales	16
2.4. Ejemplo con constraints generadas automáticamente	16
3. Implementación	17
3.1. Explicación y ejemplo de la construcción de las reglas	17
4. Experimentos	18
4.1. Aplicación de la generación automática sobre juegos simples	18
4.2. Aplicación de la generación automática sobre circomLib y comparación de constraints	18
4.3. Caso de uso, verificación y búsqueda de bugs	18

Introducción a Circom

1.1. Zero Knowledge Proof

En términos criptográficos, una prueba de conocimiento cero, conocida como Zero Knowledge Proof (o sus siglas inglesas ZKP), es una **familia de protocolos** que permite el establecimiento de métodos cuyo objetivo es permitir que una parte (**prover**) demuestre a otra (**verifier**) la validez de una afirmación **sin revelar información sobre la misma**, más allá de su validez.

Alejandro (prover) tiene el décimo de la lotería de Navidad que ha sido premiado y quiere demostrarle a **Clara (verifier)** que posee el número ganador, pero sin que ésta sea conocedora del mismo (y así evitar que pueda cobrar el premio o se lo robe). Para ello, utilizan una caja fuerte que solo se abre introduciendo la combinación del decimo premiado.

Clara escribe en un papel un dígito aleatorio, lo introduce en la caja y la cierra. Posteriormente, Clara se gira, de forma que no ve nada más de la caja.

Alejandro, que conoce el número del décimo, introduce la combinación, abriendo así la caja y leyendo el número escrito por Clara en el papel en voz alta. En ese momento, Clara sabe que ha podido abrir la caja, y por tanto, obtiene una prueba de que Alejandro es conocedor de esa información.

Sin embargo, Clara podría sospechar que Alejandro ha adivinado el dígito al azar, ya que la probabilidad es del 10 %, por lo que Clara decide repetir la prueba varias veces.

No obstante, si observamos $0,1 * 0,1 = 0,01$, se reduce la probabilidad inicial de 0,1 a 0,01. Es decir, a medida que las repeticiones aumentan, menor es la probabilidad de que se acierte al azar, ya que ésta converge a 0, alcanzando así una certeza práctica sin haber revelado ni un solo dígito del décimo.

Ejemplo 1.1: Zero Knowledge Proof

Con hemos visto en el *Ejemplo 1.1*, el concepto de **ZKP** queda ilustrado de una forma muy intuitiva. Sin embargo, para trasladar esta lógica al contexto criptográfico, es necesario formalizar estas interacciones mediante protocolos específicos.

Históricamente, el término de ZKP apareció en los años 80 como un concepto puramente teórico, hasta su desarrollo en la práctica. Dicha noción surgió con la intención de reforzar tanto la seguridad como la privacidad en el ámbito criptográfico.

La **criptografía** constituye el pilar fundamental de la seguridad en el mundo digital, permitiendo proteger información sensible frente a amenazas. Las técnicas criptográficas nos ofrecen la capacidad de garantizar confidencialidad así como un equilibrio entre transparencia y privacidad.

Gracias a este equilibrio, las técnicas criptográficas pudieron llevar a la práctica el concepto teórico ZKP en protocolos prácticos. Actualmente, esto ha llevado a la creación de grandes familias de protocolos, destacando especialmente **zk-SNARKs**.

Succinct Non-Interactive Argument of Knowledge, mayormente conocido por sus siglas **zk-SNARKs**, destacan por su reducido y concreto tamaño de prueba y corto tiempo de verificación. Sin embargo, presentan la desventaja de requerir una fase inicial conocida como **configuración de confianza**.

Definición 1.1: zk-SNARKS

Una configuración de confianza es una fase en la que se produce la generación de valores aleatorios que deben destruirse de forma inmediata. Si estos valores aleatorios son expuestos, se compromete la seguridad del sistema.

Definición 1.2: zk-SNARKS

El motivo de que estos número aleatorios deban borrarse inmediatamente es debido a que cualquier entidad conocedora de esos valores podría almacenarlos y así generar **pruebas falsas**, de forma que se podría vulnerar la seguridad del sistema.

Dada la complejidad que implica implementar estos protocolos desde cero, se han desarrollado lenguajes de programación específicos denominados **Domain Specific Languages**, conocidos como DSL.

El objetivo es que los desarrolladores que no son expertos en criptografía puedan programar las declaraciones que desean demostrar. Es en este contexto donde cobra importancia **Circom**, una herramienta creada con el fin de facilitar la creación de circuitos y su posterior transformación en sistemas de restricciones.

1.2. CIRCOM

Circom es un lenguaje de programación basado en la descripción de circuitos (DSL), cuyo objetivo principal no es el cálculo, sino definir un sistema en el que las **señales** se relacionan entre sí.

Una señal es un componente que actúa como cable dentro del circuito, el cual transporta valores a través de las puertas lógicas del mismo y que forma parte de las

ecuaciones que el programador desea verificar. Estas señales constituyen las restricciones que formarán parte del archivo **R1CS**.

Esta visión es fundamental, ya que las ZKP no pueden interpretar código directamente; requieren que la computación se descomponga en una estructura de restricciones algebraicas denominada R1CS, que detallaremos en **1.3. R1CS**.

Además de su finalidad criptográfica, es importante entender la diferencia entre un lenguaje basado en circuitos a uno de propósito general, siendo Circom basado en circuitos donde la estructura del mismo debe ser estática; esto implica que cualquier valor determinante en la estructura del circuito (como tamaños de arrays, número de iteraciones de un bucle, etc..) **debe conocerse en tiempo de compilación**. Esto se debe a que así aseguramos que el sistema de restricciones R1CS que se genera en base a este circuito, sea fijo.

El proceso de compilación de Circom refleja un paradigma híbrido entre lo **declarativo e imperativo**. De esta forma, el compilador genera dos archivos esenciales para generar la prueba:

- **Archivo R1CS:** Contiene el conjunto de todas las restricciones algebraicas que definen la lógica del circuito.
- **Programa de Testigo (Witness):** Un ejecutable (en formato C++ o WASM) encargado de calcular los valores de todas las señales del circuito, incluyendo los valores intermedios, a partir de las entradas proporcionadas.

Partiendo de estos dos archivos, podemos generar la prueba por medio de **Snarkjs**. Esta herramienta toma el archivo R1CS y el Witness obtenidos tras la compilación de un programa Circom para generar la prueba asociada. Analizaremos en detalle su funcionamiento y comandos específicos en la sección **1.4. Snarkjs**.

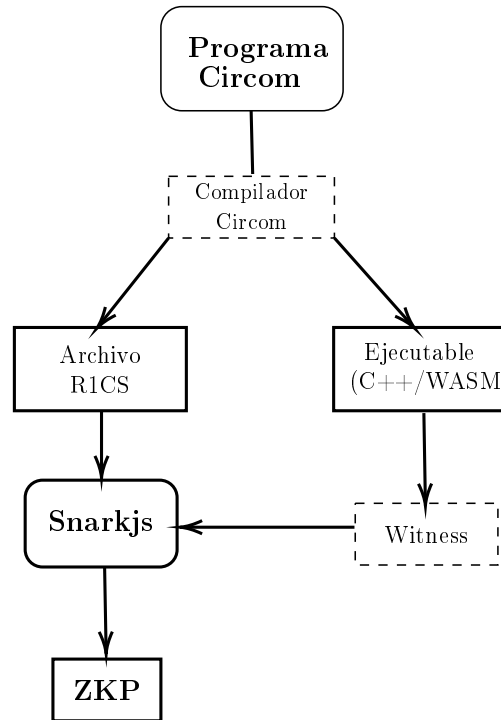
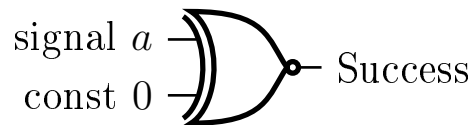


Figura 1.1: Proceso de transformación a ZKP desde Circom

Como hemos mencionado anteriormente, las señales son las restricciones establecidas por el programador, las cuáles quiere verificar. A continuación se muestra un ejemplo de un programa Circom y su representación en un circuito lógico, en el que se añade la restricción $a = 0$.

```

1      template var_zero(){
2          signal input a;
3          a * 1 == 0;
4      }
5
6      main component = var_zero();
  
```



Ejemplo 1.2: Archivo var_zero.circom

1.3. R1CS

Rank-1 Constraint System (conocido como **R1CS**), es un sistema de representación matemático diseñado para transformar tareas computacionales complejas en un conjunto de restricciones algebraicas. Su relevancia incide en la propiedad de que generar la solución a un problema puede ser caro, mientras que su **verificación es muy eficiente y rápida**, clave en las pruebas de conocimiento cero.

R1CS impone restricciones matemáticas complejas. Las ecuaciones, aparte de no poder exceder el segundo grado (*condición necesaria, pero no suficiente*), cada restricción debe poder expresarse como el producto de dos combinaciones lineales de variables, teniendo como resultado una tercera combinación lineal:

$$A \cdot B - C = 0$$

Donde A , B y C son combinaciones lineales de las señales del circuito. En términos prácticos, esto implica que una sola restricción solo puede contener una multiplicación entre variables. Operaciones que incluyan múltiples productos independientes, aunque sean de segundo grado, deben ser descompuestas en varias restricciones consecutivas por medio de variables auxiliares con el fin de cumplir con este estándar.

Siguiendo el *Ejemplo 1.2*, la restricción $a * 1 == 0$ se traduce al formato R1CS como el producto de dos combinaciones lineales A y B , forzando así que el valor de la señal a sea nulo.

Dada la declaración:

$$(a) \times (1) = (0)$$

Donde la estructura $A \cdot B = C$ se satisface asignando:

- $A = a$ (Combinación lineal 1)
- $B = 1$ (Combinación lineal 2)
- $C = 0$ (Resultado de la restricción)

Ejemplo 1.3: Transformación de `var_zero.circom` a formato R1CS

Gracias a R1CS, es posible que el código **escrito en Circom pueda transformarse en un sistema de restricciones**, consiguiendo así que la verificación sea sumamente eficiente.

1.4. Snarkjs

Snarkjs es una librería de JavaScript creada para la generación y verificación de pruebas de conocimiento cero partiendo de las restricciones en formato R1CS correspondientes, que puede ser ejecutado tanto en dispositivos como en servidores con *Node.js*, materializando el concepto de *1.1 zk-SNARKS* nombrado anteriormente.

Como se mostró previamente en la *1.1 Figura*, Snarkjs requiere de dos archivos para generar la prueba:

- *Witness*, generado por el ejecutable C++/WASM.
- *Archivo R1CS*, restricciones de la lógica del circuito.

Una vez disponibles estos dos archivos y la *1.2 Configuración de confianza* finalizada, Snarkjs genera la prueba de conocimiento cero que permite que el Prover pueda demostrarle al Verifier la validez de su declaración.

1.5. Descripción del problema

Desarrollo del problema

2.1. Ejemplo del problema

2.2. Notación

A lo largo del documento estudiaremos cómo traducir instrucciones y expresiones generadas por un lenguaje imperativo simple en un conjunto de ecuaciones en formato R1CS equivalente definimos las siguientes funciones:

Definimos la función de traducción de expresiones $TC : Expr \rightarrow C \times Expr$ siendo:

- C un sistema de ecuación en formato R1CS
- C es una expresión lineal

Definición 2.1: Notación

La intuición es que en caso de que las constraints C se verifiquen, entonces e y e' siempre tienen el mismo valor.

Vamos a definir la función TC de forma inductiva, comenzando por los casos base (números y variables), para luego pasar a definir los casos compuestos.

Casos base

- Si la expresión es un número n :

$$TC(n) = TC(\emptyset, n)$$

- Si la expresión es una variable v :

$$TC(v) = TC(\emptyset, v)$$

2.3. Reglas de transformación

En esta sección se encuentra la documentación relativa a como se han implementado las reglas de transformación pertinentes para hacer posible la migración del problema

descrito a la práctica.

2.3.1. Operadores Lógicos

Operación	Deducción
$e = \text{not } b$	$\frac{TC(e_1) = (C, e'_1)}{TC(\text{not } e_1) = ((e'_1) * (e'_1 - 1) == 0) \wedge (C, 1 - e'_1)}$
$e = \text{and } b$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \text{ and } e_2) = \left(\begin{array}{c} C_1 \wedge C_2 \wedge \\ (e'_1 * (e'_1 - 1) = 0) \wedge (e'_2 * (e'_2 - 1) = 0) \\ \wedge \{v_{aux} = e'_1 * e'_2\}, v_{aux} \end{array} \right)}$
$e = \text{or } b$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \text{ or } e_2) = \left(\begin{array}{c} C_1 \wedge C_2 \wedge \\ (e'_1 * (e'_1 - 1) = 0) \wedge (e'_2 * (e'_2 - 1) = 0) \\ \wedge \{v_{aux} = e'_1 * e'_2\}, e'_1 + e'_2 - v_{aux} \end{array} \right)}$
$e = e_1 \& e_2$	$\frac{TC(e_1) = (C_1, e_{1-\text{binario}}) \quad TC(e_2) = (C_2, e_{2-\text{binario}})}{TC(e_1 \& e_2) = (C_1 \wedge C_2 \wedge \{v_{aux} = e_{1-\text{binario}}[i] * e_{2-\text{binario}}[i]\}, v_{aux})}$
$e = e_1 \mid e_2$	$\frac{TC(e_1) = (C_1, e_{1-\text{binario}}) \quad TC(e_2) = (C_2, e_{2-\text{binario}})}{TC(e_1 \mid e_2) = (C_1 \wedge C_2 \wedge \{v_{aux} = e_{1-\text{binario}}[i] * e_{2-\text{binario}}[i]\}, e_{1-\text{binario}}[i] + e_{2-\text{binario}}[i] - v_{aux})}$

2.3.2. Operadores de Comparación

Tratamiento de número negativos

Un detalle a observar en las funciones $LessThan()$ y $LessThanEq()$ es que para números negativos podría dificultar su uso.

Esto se debe a que en el contexto de Circom trabajamos sobre un cuerpo finito $\mathbb{F}_p$, de forma que los números negativos realmente no existen. En su lugar, se representan mediante el módulo del campo.

Por ejemplo, el valor -1 se interpreta como $p-1$, un número extremadamente grande que se aproxima al orden del primo del campo.

Para solventar este problema, recurrimos a la transformación de los números a comparar de sistema decimal a binario. Esta conversión es estrictamente necesaria para asegurar que los números correspondientes no exceden el rango especificado.

Num2Bits(n) consigue cumplir esta precondition ya que su funcionamiento reside en la transformación de decimal a binario, reconstruyendo el número decimal entrante durante la misma. De esta forma, al terminar la transformación y antes de salir de la función, se realiza una comprobación para verificar si el número decimal entrante y el número decimal reconstruido son iguales.

Al ser el número entrante excesivamente grande y superar el rango establecido, nunca se cumplirá la restricción de que ambos sean iguales debido a que el tamaño máximo fijado N es menor que el del número negativo, y por tanto, la reconstrucción no realizará un bucle de tamaño mayor a N .

De esta forma, no habrá un bucle que reconstruya un número decimal negativo ya que nunca se realizan las iteraciones necesarias, forzando de esta forma a que solo los números positivos cumplan dicha condición.

Intuición del funcionamiento de *Num2Bits(n)* con un número negativo entrante:

- El número negativo se interpreta como el módulo del campo.
- Éste se interpretará como un número superior a $2^n - 1$, saliéndose fuera del rango $[0, 2^n - 1]$, por tanto la constraints no se pueden cumplir y la prueba no se generará.
- En conclusión, con *Num2Bits()* garantizamos el uso de número positivos dentro del rango mencionado de manera implícita.

Observación: Gracias a la constraint *lc1 === in* (fuerza a que la entrada cumpla el rango $[0, 2^n - 1]$), podemos evitar el uso de número negativos, ya que estos no cumplirán tal restricción.

Num2Bits(n):

```

1      template Num2Bits(n){
2          signal input in;
3          signal output out[n];
4          var lc1=0; // Reconstruye el numero decimal a
                    convertir para la constraint
5
6          var e2=1; // Evitar la potencia
7          for (var i = 0; i<n; i++) {
8              out[i] <-- (in >> i) & 1; // Contruimos el numero en
                    binario
9              out[i] * (out[i] -1 ) === 0;
10             lc1 += out[i] * e2;
11             e2 = e2+e2;
12         }
13
14         lc1 === in; // Constraint mencionada anteriormente
15     }
```

Para la implementación de los operadores de comparación relativos a $<, \leq, >$ y $\geq$ únicamente es necesaria la función *SecureLessThanEq*(n).

El motivo de esto es que para una comparación entre una variable a y b , con *SecureLessThanEq*(n) podemos conocer si $a \leq b$ o bien $a > b$ en caso contrario (*GreaterThan*(n)).

Siguiendo esta lógica, podemos usar la misma función invirtiendo el orden de los parámetros, consiguiendo de esta manera los operadores *GreaterThanEq*(n) y *LessThan*(n) si $b \leq a$ o $b > a$.

La implementación de *SecureLessThanEq*(n), aparte de utilizar la función *Num2Bits*(n) para cada par de variables a comparar, aplica Complemento a 2.

Es decir, dadas las variables A y B , previamente transformadas a bits, aplicamos $(\text{NOT } A) + B + 1$. La inversión de los bits de A y la suma tanto de B como de 1 es equivalente a la operación $B - A$.

Por tanto, cuando el minuendo es mayor o igual que el sustraendo, la suma de los bits junto con el complemento a dos produce un acarreo (overflow) en la posición n (el bit extra, bit más significativo). Por el contrario, si el minuendo es menor que el sustraendo, no se produce dicho overflow.

De esta forma sabemos que si el bit de acarreo es 1, podemos concluir que $B \geq A$, ya que al hacer la suma con Complemento a 2, tenemos que:

$$\begin{aligned}
 & (\text{NOT } A) + B + 1 \\
 & [(2^n - 1) - A] + B + 1 \\
 & 2^n - 1 - A + B + 1 \\
 & 2^n - A + B \\
 & B + (2^n - A)
 \end{aligned} \tag{2.1}$$

Como podemos ver en las ecuaciones anteriores, debido al requerimiento de $2^n - 1$ por parte de $\text{NOT } A$, es vital el $+1$ de $(\text{NOT } A) + B + 1$. De lo contrario el resultado de la resta estaría desplazado por una unidad al no eliminar ese -1 de $\text{NOT } A$.

Partiendo de $2^n + B - A$, si B es mayor o igual que A , entonces $(B-A) \geq 0$. De este modo, tendríamos que $2^n + (\text{num.positivo}) \geq 2^n$, concluyendo que si $B \geq A$ obtendríamos siempre un resultado mayor o igual a 2^n , excediendo siempre los n bits y como consecuencia, produciendo un overflow.

Siguiendo la misma lógica, si el bit de acarreo es 0 significa que A es mayor que B ya que $B-A < 0$, y de esta forma obtendríamos $2^n + (\text{num.positivo}) < 2^n$, concluyendo así que no podría producirse un overflow.

```

template SecureLessThan(n){
    signal input in[2];
    signal output out;

    //-----

    component checkA = Num2Bits(n);
    checkA.in <== in[0];

    component checkB = Num2Bits(n);
    checkB.in <== in[1];

    adder = BinSum(n, 2);

    var i;
    for (i=0;i<n;i++) {
        checkA.out[i] ==> adder.in[0][i];
        checkB.out[i] ==> adder.in[1][i];
    }

    // Miramos el bit + significativo (
    // MSB), 1 si A < B, 0 de lo
    // contrario
    adder.out[n-1] ==> out;
}

```

Con esta pequeña modificación, al usar para ambos números a comparar la función *Num2Bits()* nos aseguramos de que se cumpla la restricción de ser números enteros positivos dentro del rango.

Tamaño de los números comparados

Por otro lado, es importante el detalle de que podemos comparar dos números de tamaño conocido e igual, mientras que puede que comparemos dos números cuyo tamaño no conocemos o cuyos tamaños son diferentes.

Para resolver este problema, vamos a definir dos operadores de comparación por cada operador de comparación existente exceptuando la igualdad y desigualdad.

Un ejemplo de cuáles serían estos dos operadores sería el siguiente, para el operador $\leq$:

Operador $\leq$

- $\leq$: Comparaciones de cuyos números se conoce el tamaño y además tienen el mismo tamaño.
- La llamada a la función sería *lessThan(253)* para estos casos, ya que el tamaño máximo es 253 en el cuerpo finito.

Operador $\leq_n$

- $\leq_n$: Comparaciones de cuyos números no se conoce el tamaño y/o no tiene el mismo tamaño.

- La llamada a la función sería *lessThan*(*n*) para estos casos.

A continuación, definiremos para cada operador su deducción.

Operación	Deducción
$e = e_1 = e_2$	$\frac{\text{signal diff} \leq e_1 - e_2 \quad TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 = e_2) = (C_1 \wedge C_2 \wedge \{IsZero().in = diff, IsZero().out = v_{aux}\}, v_{aux})}$
$e = e_1 \neq e_2$	$\frac{\text{signal diff} \leq e_1 - e_2 \quad TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \neq e_2) = (C_1 \wedge C_2 \wedge \{IsZero().in = diff, IsZero().out = v_{aux}\}, 1 - v_{aux})}$
$e = e_1 < e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 < e_2) = \left(C_1 \wedge C_2 \wedge \begin{array}{l} \{SecureLessThan(253).in1 = e'_1, \\ SecureLessThan(253).in2 = e'_2, \\ SecureLessThan(253).out = v_{aux}\}, \end{array} v_{aux} \right)}$
$e = e_1 <_n e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 <_n e_2) = \left(C_1 \wedge C_2 \wedge \begin{array}{l} \{SecureLessThan(n).in1 = e'_1, \\ SecureLessThan(n).in2 = e'_2, \\ SecureLessThan(n).out = v_{aux}\}, \end{array} v_{aux} \right)}$
$e = e_1 \leq e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \leq e_2) = \left(C_1 \wedge C_2 \wedge \begin{array}{l} \{SecureLessThanEq(253).in1 = e'_1, \\ SecureLessThanEq(253).in2 = e'_2, \\ SecureLessThanEq(253).out = v_{aux}\}, \end{array} v_{aux} \right)}$
$e = e_1 \leq_n e_2$	$\frac{TC(e_1) = (C_1, e'_1) \quad TC(e_2) = (C_2, e'_2)}{TC(e_1 \leq_n e_2) = \left(C_1 \wedge C_2 \wedge \begin{array}{l} \{SecureLessThanEq(n).in1 = e'_1, \\ SecureLessThanEq(n).in2 = e'_2, \\ SecureLessThanEq(n).out = v_{aux}\}, \end{array} v_{aux} \right)}$
$e = e_1 > e_2$	

Nota: Las funciones `IsZero()` son circuitos que comparan si un valor es igual a 0.
Nota: Las funciones `IsEqual()` comprueba si dados dos números, estos son iguales o no.

2.3.3. Operadores Aritméticos

2.3.4. Operadores de desplazamiento

2.3.5. Acceso a Arrays

2.3.6. Expresiones condicionales

2.3.7. Expresiones condicionales

2.4. Ejemplo con constraints generadas automáticamente

Implementación

3.1. Explicación y ejemplo de la construcción de las reglas

Experimentos

- 4.1. Aplicación de la generación automática sobre juegos simples
- 4.2. Aplicación de la generación automática sobre circomLib y comparación de constraints
- 4.3. Caso de uso, verificación y búsqueda de bugs